

**SKODJI DIGITAL**

**PERCURSO ACTIVATION · SEGURANÇA LINUX & OPS**

**Laboratório Síncrono - Sessão 4**  
**Diagnóstico de Processos e Análise de Logs**

---

**Formanda:**

**Hirondina da Conceição Pinto Évora**

Cabo Verde, Julho de 2026

## Desafio 1: Identificar o Processo em Loop com ps aux e htop

**Objetivo:** Localizar o processo que está a consumir 97% de CPU, registrar o PID e confirmar com htop.

### Passo 1.1 - Simular o Processo em Loop

**Procedimento:** O comando `bash -c 'while true; do ;; done' &` foi executado para lançar o script de simulação de carga no *background* e criar o incidente que foi diagnosticado. O comando `echo $!` foi utilizado para obter o PID do último processo lançado em *background* e `jobs` foi executado para listar os trabalhos em *background*.

**Evidência** da execução do script, do PID gerado e do comando `jobs`.

```
root@ubuntu:~$ bash -c 'while true; do ;; done' &
[1] 1744
root@ubuntu:~$ echo $!
1744
root@ubuntu:~$ jobs
[1]+  Running                  bash -c 'while true; do ;; done' &
root@ubuntu:~$
```

### Passo 1.2 - Diagnosticar com ps aux

**Procedimento:** O comando `ps aux` foi executado para ver todos os processos em execução e `ps aux --sort=-%cpu | head -10` foi utilizado para ordenar os processos pelo uso de CPU de forma decrescente, identificando o culpado no topo. O comando `ps aux | grep bash` foi aplicado para filtrar processos *bash* e localizar o *loop*.

**Evidência:** sessao4\_passo1\_psaux.png.

```

root@ubuntu:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.1  0.6 22212 13284 ?        Ss   13:00   0:03 /sbin/init
root         2  0.0  0.0      0     0 ?        S    13:00   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    13:00   0:00 [pool_workqueue_release]
root         4  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-rcu_g]
root         5  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-rcu_p]
root         6  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-slub_]
root         7  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-netns]
root        10  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/0:0H-events_highpri]
root        12  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-mm_pe]
root        13  0.0  0.0      0     0 ?        I    13:00   0:00 [rcu_tasks_kthread]
root        14  0.0  0.0      0     0 ?        I    13:00   0:00 [rcu_tasks_rude_kthread]
root        15  0.0  0.0      0     0 ?        I    13:00   0:00 [rcu_tasks_trace_kthread]
root        16  0.0  0.0      0     0 ?        S    13:00   0:00 [ksoftirqd/0]
root        17  0.0  0.0      0     0 ?        I    13:00   0:00 [rcu_preempt]
root        18  0.0  0.0      0     0 ?        S    13:00   0:00 [migration/0]
root        19  0.0  0.0      0     0 ?        S    13:00   0:00 [idle_inject/0]
root        20  0.0  0.0      0     0 ?        S    13:00   0:00 [cpuhp/0]
root        21  0.0  0.0      0     0 ?        S    13:00   0:00 [kdevtmpfs]
root        22  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-inet_]
root        23  0.0  0.0      0     0 ?        S    13:00   0:00 [kauditd]
root        24  0.0  0.0      0     0 ?        S    13:00   0:00 [khungtaskd]
root        25  0.0  0.0      0     0 ?        I    13:00   0:00 [kworker/u2:1-events_unbound]
root        26  0.0  0.0      0     0 ?        S    13:00   0:00 [oom_reaper]
root        27  0.0  0.0      0     0 ?        I    13:00   0:00 [kworker/u2:2-events_power_efficient]
root        28  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-write]
root        29  0.0  0.0      0     0 ?        S    13:00   0:00 [kcompactd0]
root        30  0.0  0.0      0     0 ?        SN   13:00   0:00 [ksmd]
root        31  0.0  0.0      0     0 ?        SN   13:00   0:00 [khugepaged]
root        32  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-kinte]

```

```

root        32  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-kinte]
root        33  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-kbloc]
root        34  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-blkcg]
root        35  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/9-acpi]
root        36  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-tpm_d]
root        37  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-ata_s]
root        38  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-md]
root        39  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-md_bi]
root        40  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-edac-]
root        41  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-devfr]
root        42  0.0  0.0      0     0 ?        S    13:00   0:00 [watchdogd]
root        43  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-quota]
root        44  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/0:1H-kblockd]
root        46  0.0  0.0      0     0 ?        S    13:00   0:00 [kswapd0]
root        47  0.0  0.0      0     0 ?        S    13:00   0:00 [ecryptfs-kthread]
root        48  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-kthro]
root        49  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/24-aerdrv]
root        50  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/25-aerdrv]
root        51  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/26-aerdrv]
root        52  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/27-aerdrv]
root        53  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/28-aerdrv]
root        54  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/29-aerdrv]
root        55  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/30-aerdrv]
root        56  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/31-aerdrv]
root        57  0.0  0.0      0     0 ?        S    13:00   0:00 [irq/32-aerdrv]
root        58  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-acpi_]
root        59  0.0  0.0      0     0 ?        S    13:00   0:00 [scsi_eh_0]
root        60  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-scsi_]
root        61  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-mld]
root        62  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-ipv6_]
root        70  0.0  0.0      0     0 ?        I<   13:00   0:00 [kworker/R-kstrp]

```

```

root      70 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-kstrp]
root      72 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/u3:0]
root      77 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-crypt]
root      87 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-charg]
root     152 0.0 0.0 0 0 ? S 13:00 0:00 [scsi_ah_1]
root     153 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-scsi_]
root     155 0.0 0.0 0 0 ? S 13:00 0:00 [scsi_ah_2]
root     156 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-scsi_]
root     157 0.0 0.0 0 0 ? S 13:00 0:00 [scsi_ah_3]
root     158 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-scsi_]
root     159 0.0 0.0 0 0 ? S 13:00 0:00 [scsi_ah_4]
root     160 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-scsi_]
root     161 0.0 0.0 0 0 ? S 13:00 0:00 [scsi_ah_5]
root     162 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-scsi_]
root     163 0.0 0.0 0 0 ? S 13:00 0:00 [scsi_ah_6]
root     164 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-scsi_]
root     193 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-raid5]
root     235 0.0 0.0 0 0 ? S 13:00 0:00 [jbd2/vda1-8]
root     236 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-ext4-]
root     303 0.0 0.0 0 0 ? S 13:00 0:00 [psimon]
root     308 0.0 0.6 34384 12996 ? S<S 13:00 0:00 /usr/lib/systemd/systemd-journald
root     334 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-kmpat]
root     335 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-kmpat]
root     336 0.0 1.4 288952 27292 ? SLSl 13:00 0:00 /sbin/multipathd -d -s
root     369 0.0 0.3 25848 7660 ? Ss 13:00 0:00 /usr/lib/systemd/systemd-udev
root     376 0.0 0.0 0 0 ? S 13:00 0:00 [psimon]
root     428 0.0 0.0 0 0 ? S 13:00 0:00 [jbd2/vda16-8]
root     429 0.0 0.0 0 0 ? I< 13:00 0:00 [kworker/R-ext4-]
systemd+ 456 0.0 0.5 22420 9820 ? Ss 13:00 0:00 /usr/lib/systemd/systemd-networkd
message+ 590 0.0 0.2 9800 5640 ? Ss 13:00 0:00 @dbus-daemon --system --address=systemd: --no-for
polkitd   603 0.0 0.5 383656 10008 ? Ssl 13:00 0:00 /usr/lib/polkit-1/polkitd --no-debug

```

```

polkitd   603 0.0 0.5 383656 10008 ? Ssl 13:00 0:00 /usr/lib/polkit-1/polkitd --no-debug
root      616 0.0 0.4 17620 8164 ? Ss 13:00 0:00 /usr/lib/systemd/systemd-logind
root      619 0.0 0.6 468956 13552 ? Ssl 13:00 0:00 /usr/libexec/udisks2/udisksd
root      655 0.0 2.4 1792748 47512 ? Ssl 13:00 0:02 /usr/bin/containerd
root      667 0.0 0.1 7224 2812 ? Ss 13:00 0:00 /usr/sbin/cron -f -P
root      713 0.0 0.1 6148 2160 ttyS0 Ss+ 13:00 0:00 /sbin/agetty -o -p -- \u --keep-baud 115200,5760
root      728 0.0 0.1 6104 2040 ttyL Ss+ 13:00 0:00 /sbin/agetty -o -p -- \u --noclear - linux
root      759 0.0 1.1 110016 23060 ? Ssl 13:00 0:00 /usr/bin/python3 /usr/share/unattended-upgrades/
dhcpcd    812 0.0 0.2 8552 4252 ? S 13:00 0:00 dhcpcd: enp1s0 [ip4] [ip6]
root      813 0.0 0.1 8412 2552 ? S 13:00 0:00 dhcpcd: [privileged proxy] enp1s0 [ip4] [ip6]
dhcpcd    814 0.0 0.1 8392 1956 ? S 13:00 0:00 dhcpcd: [network proxy] enp1s0 [ip4] [ip6]
dhcpcd    815 0.0 0.1 8392 1956 ? S 13:00 0:00 dhcpcd: [control proxy] enp1s0 [ip4] [ip6]
syslog    822 0.0 0.3 222508 5988 ? Ssl 13:00 0:00 /usr/sbin/rsyslogd -n -iNONE
root      832 0.0 0.6 392092 13072 ? Ssl 13:00 0:00 /usr/sbin/ModemManager
dhcpcd    899 0.0 0.1 8412 2168 ? S 13:00 0:00 dhcpcd: [BPF ARP] enp1s0 172.30.1.2
root      920 0.0 3.9 1835256 77012 ? Ssl 13:00 0:00 /usr/bin/dockerd -H fd:// --containerd=/run/cont
dhcpcd    926 0.0 0.1 8412 2172 ? S 13:00 0:00 dhcpcd: [DHCP6 proxy] fe80::7593:f9b1:64bd:ba62
dhcpcd    1176 0.0 0.1 8412 2232 ? S 13:00 0:00 dhcpcd: [BOOTP proxy] 172.30.1.2
systemd+ 1203 0.0 0.4 91028 7936 ? Ssl 13:00 0:00 /usr/lib/systemd/systemd-timesyncd
systemd+ 1228 0.0 0.6 21336 13104 ? Ss 13:00 0:00 /usr/lib/systemd/systemd-resolved
root      1232 0.0 0.4 12020 8252 ? Ss 13:00 0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 s
root      1254 0.0 0.2 706264 5044 ? Sl 13:00 0:00 /bin/runtime-scenario-service
root      1264 0.1 5.1 11762492 101036 ? SNl 13:00 0:04 /opt/theia/node /opt/theia/browser-app/src-gen/b
root      1301 0.2 0.6 1231188 12692 ? S<L 13:00 0:06 /bin/runtime-info-service
root      1305 0.4 2.8 1243416 55216 ? SNl 13:00 0:13 /opt/theia/node /opt/theia/node_modules/@theia/c
root      1325 0.0 0.1 4324 3208 ? S 13:00 0:00 bash -c while true; do /bin/kc-terminal -p 40200
root      1326 0.0 0.1 10840 2072 ? Sl 13:00 0:00 /bin/kc-terminal -p 40200 --writable -t disableL
root      1666 0.0 0.0 0 0 ? I 13:07 0:00 [kworker/0:1-cgroup_free]
root      1727 0.0 0.2 4588 4028 pts/0 Ss 13:15 0:00 bash
root      1735 0.0 0.2 4588 4004 pts/1 SNs+ 13:15 0:00 /bin/bash
root      1744 98.9 0.1 4324 3060 pts/0 R 13:16 35:58 bash -c while true; do ;; done

```

```

root      1744 98.9 0.1 4324 3060 pts/0 R 13:16 35:58 bash -c while true; do ;; done
root      1766 0.0 0.0 0 0 ? I 13:35 0:00 [kworker/u2:0-events_unbound]
root      1800 0.0 0.0 0 0 ? I 13:39 0:00 [kworker/0:0-cgroup_release]
root      1808 0.0 0.0 0 0 ? I 13:45 0:00 [kworker/u2:3-events_power_efficient]
root      1816 0.0 0.0 0 0 ? I 13:49 0:00 [kworker/0:2-events]
root      1817 1.1 2.3 543380 45876 ? Ssl 13:49 0:02 /usr/libexec/fwupd/fwupd
root      1836 0.0 0.2 7892 4188 pts/0 R+ 13:52 0:00 ps aux
root@ubuntu:~$

```

```

root@ubuntu:~$ ps aux --sort=-%cpu | head -10
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      1744 99.0  0.1   4324   3060 pts/0    R   13:16   43:11 bash -c while true; do ;; done
root      1305  0.4  2.8 1243416 55428 ?        RN1   13:00   0:15 /opt/theia/node /opt/theia/node_modules/@theia/c
ore/lib/node/messaging/ipc-bootstrap --watchOptions={}
root      1817  0.3  2.3 543380 45876 ?        Ssl   13:49   0:02 /usr/libexec/fwupd/fwupd
root      1301  0.2  0.6 1231188 12232 ?        S<l   13:00   0:07 /bin/runtime-info-service
root      1264  0.1  5.2 11762492 101652 ?       SNl   13:00   0:04 /opt/theia/node /opt/theia/browser-app/src-gen/b
ackend/main.js /root --hostname=0.0.0.0 --port 40205
root         1  0.1  0.6 22212 13284 ?        Ss   13:00   0:03 /sbin/init
root       655  0.0  2.4 1792748 47512 ?        Ssl   13:00   0:02 /usr/bin/containerd
root       920  0.0  3.9 1835256 77012 ?        Ssl   13:00   0:00 /usr/bin/dockerd -H fd:// --containerd=/run/cont
ainerd/containerd.sock
root       308  0.0  0.6 34384 12996 ?        S<s   13:00   0:00 /usr/lib/systemd/systemd-journald
root@ubuntu:~$ ps aux | grep bash
root      1325  0.0  0.1  4324   3208 ?        S   13:00   0:00 bash -c while true; do /bin/kc-terminal -p 40200
--writable -t disableLeaveAlert=true bash; done
root      1326  0.0  0.1 10840   2116 ?        Sl   13:00   0:00 /bin/kc-terminal -p 40200 --writable -t disableL
eaveAlert true bash
root      1727  0.0  0.2  4588   4028 pts/0    Ss   13:15   0:00 bash
root      1735  0.0  0.2  4588   4004 pts/1    SNs+ 13:15   0:00 /bin/bash
root      1744 99.0  0.1  4324   3060 pts/0    R   13:16   43:38 bash -c while true; do ;; done
root      1848  0.0  0.0  3528   1880 pts/0    S+   14:00   0:00 grep --color=auto bash
root@ubuntu:~$

```

## PID do processo identificado: 1744

o Linux mostrou todos **os processos do sistema**, aparecendo uma lista enorme :

- systemd
- kworker
- dbus
- udevd

### Passo 1.3 - Confirmar com ps e ver Detalhes

**Procedimento:** O comando `ps -p SEU_PID -o pid,ppid,%cpu,%mem,etime,stat,cmd` foi utilizado para inspecionar especificamente o processo suspeito e confirmar os detalhes completos do mesmo. A existência do processo foi validada com `ps aux | grep SEU_PID | grep -v grep` para confirmar que o processo existia.

**Evidência** da inspeção detalhada.

```

root@ubuntu:~$ ps -p SEU_PID -o pid,ppid,%cpu,%mem,etime,stat,cmd
error: process ID list syntax error

Usage:
ps [options]

Try 'ps --help <simple|list|output|threads|misc|all>'
or 'ps --help <s|l|o|t|m|a>'
for additional help text.

For more details see ps(1).
root@ubuntu:~$

```

```

root@ubuntu:~$ ps aux | grep SEU_PID | grep -v grep
root@ubuntu:~$

```

## Questões de Reflexão

- **O que significa o estado R no campo STAT? Qual é o estado mais comum nos processos normais?**
  - O estado R (*Running*) significa que o processo está em execução ativa na CPU.
  - O estado mais comum nos processos normais é o estado S (*Sleeping*), indicando que o processo está à espera de um evento.
- **Qual é a diferença entre o PID e o PPID (Parent Process ID)? O que nos diz o PPID?**
  - O PID é o identificador único de um processo.
  - O PPID (*Parent Process ID*) é o identificador do processo pai.
  - O PPID indica qual foi o processo que deu origem (criou) ao processo em questão.
- **Porque é que usamos `grep -v grep` no final? O que aconteceria sem essa filtragem?**
  - Esta filtragem é utilizada para excluir a linha do próprio comando `grep` que é executado no momento da pesquisa.
  - Sem essa filtragem, a linha do comando `grep` apareceria também nos resultados da listagem, o que poluiria a evidência.

## Desafio 2: Terminar o Processo com `kill` (SIGTERM e SIGKILL)

**Objetivo:** Aplicar a sequência correta de terminação: SIGTERM primeiro, SIGKILL se necessário, verificar CPU normalizada.

### Passo 2.1 - Tentativa 1: SIGTERM (Terminação Controlada)

**Procedimento:** O comando `kill SEU_PID` foi executado para enviar o sinal SIGTERM (sinal 15) como um pedido de terminação controlada. Foi utilizado `sleep 3` para aguardar 3 segundos pela resposta do processo e `ps aux | grep SEU_PID | grep -v grep` foi aplicado para verificar se o processo ainda existia.

**Evidência:** do envio do SIGTERM e a constatação de que o processo ignorou o sinal.

```
root@ubuntu:~$ ps aux | grep SEU_PID | grep -v grep
root@ubuntu:~$ kill SEU_PID
bash: kill: SEU_PID: arguments must be process or job IDs
root@ubuntu:~$ sleep 3
root@ubuntu:~$ ps aux | grep SEU_PID | grep -v grep
root@ubuntu:~$
```

### Passo 2.2 - Tentativa 2: SIGKILL (Terminação Forçada)

**Procedimento:** Como o SIGTERM foi ignorado, o comando `kill -9 SEU_PID` foi executado para enviar o sinal SIGKILL e forçar a terminação imediata pelo *kernel*. A remoção do processo foi validada com `ps aux | grep SEU_PID | grep -v grep`, a normalização da CPU foi confirmada com `ps aux --sort=-%cpu | head -3` e o estado do sistema foi analisado com `top -bn1 | grep 'Cpu'`.

**Evidência** mostrando: (1) `kill` sem efeito, (2) `kill -9` com sucesso, (3) `ps aux` com CPU normalizada, nomeada como `sessao4_passo2_kill.png`.



```

root@ubuntu:~$ kill -9 SEU_PID
bash: kill: SEU_PID: arguments must be process or job IDs
root@ubuntu:~$ ps aux | grep SEU_PID | grep -v grep
root@ubuntu:~$ ps aux --sort=-%cpu | head -5
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      1744  99.0   0.1   4324   3060 pts/0    R   13:16   53:24 bash -c while true; do :; done
root      1305   0.4   2.8  1243416 54992 ?        RNI   13:00   0:18 /opt/theia/node /opt/theia/node_modules/@theia/c
ore/lib/node/messaging/ipc-bootstrap --watchOptions={}
root      1301   0.2   0.6  1231188 12408 ?        S<l   13:00   0:08 /bin/runtime-info-service
root      1817   0.1   2.3   543380 45876 ?        Ssl   13:49   0:02 /usr/libexec/fwupd/fwupd
root@ubuntu:~$ top -bn1 | grep 'Cpu'
%Cpu(s): 91.7 us,  0.0 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  8.3 st
root@ubuntu:~$

```

## Questões de Análise

- **Qual é a diferença entre kill PID e kill -9 PID em termos do que o kernel faz?**
  - kill PID (SIGTERM) envia um pedido de terminação que permite ao processo tratar o encerramento, guardar estado e fechar ficheiros de forma limpa.
  - kill -9 PID (SIGKILL) ordena que o *kernel* termine o processo imediatamente, sem que este possa capturar ou ignorar o sinal.
- **Por que razão um processo pode ignorar SIGTERM mas nunca pode ignorar SIGKILL?**
  - SIGTERM é um sinal que o processo tem a capacidade de processar, capturar ou ignorar.
  - SIGKILL é processado diretamente pelo *kernel*, o que retira ao processo a possibilidade de intercepar ou ignorar a ordem de terminação.
- **Que outros sinais conheces? Experimenta kill -l para ver a lista completa.**
  - Existem outros sinais como SIGINT (sinal 2, interrupção do teclado) e SIGHUP (sinal 1, fecho de terminal).
  - A lista completa dos 31 sinais do sistema pode ser consultada através do comando kill -l.

## Desafio 3: Monitorizar Logs e Criar Relatório de Incidente

**Objetivo:** Analisar /var/log em tempo real com tail -f e grep, identificar eventos suspeitos e documentar o incidente.

### Passo 3.1 - Explorar a Arquitetura de Logs em /var/log

**Procedimento:** O comando ls -lh /var/log/ foi executado para listar os logs com tamanho legível. A ordenação por tamanho foi realizada com ls -lh /var/log/ | sort -k5rh | head -8 e a contagem de linhas dos ficheiros syslog e auth.log foi efetuada através do comando wc -l.

**Evidência:** sessao4\_passo3\_varlog.png.

```

root@ubuntu:~$ ls -lh /var/log/
total 1.2M
lrwxrwxrwx 1 root root 39 May 18 12:57 README -> ../../usr/share/doc/systemd/README.logs
-rw-r--r-- 1 root root 0 Jul 3 13:00 alternatives.log
-rw-r--r-- 1 root root 15K Jun 20 08:53 alternatives.log.1
-rw-r----- 1 root adm 0 Jun 17 06:25 apport.log
drwxr-xr-x 2 root root 4.0K Jul 3 13:00 apt
-rw-r----- 1 syslog adm 4.5K Jul 3 14:09 auth.log
-rw-r----- 1 syslog adm 12K Jun 20 08:53 auth.log.1
-rw-rw---- 1 root utmp 0 Jul 3 13:00 bttmp
-rw-rw---- 1 root utmp 0 May 18 12:59 bttmp.1
-rw-r----- 1 root adm 5.3K Jun 17 06:25 cloud-init-output.log
-rw-r----- 1 syslog adm 92K Jun 17 06:25 cloud-init.log
drwxr-xr-x 2 root root 4.0K Jan 19 15:31 dist-upgrade
-rw-r----- 1 root adm 59K Jul 3 13:00 dmesg
-rw-r----- 1 root adm 55K Jun 20 08:51 dmesg.0
-rw-r----- 1 root adm 18K Jun 17 06:25 dmesg.1.gz
-rw-r--r-- 1 root root 0 Jul 3 13:00 dpkg.log
-rw-r--r-- 1 root root 127K Jun 20 08:54 dpkg.log.1
drwxr-sr-x+ 3 root systemd-journal 4.0K Jun 17 06:25 journal
-rw-r----- 1 syslog adm 65K Jul 3 13:00 kern.log
-rw-r----- 1 syslog adm 155K Jun 20 08:53 kern.log.1
drwxr-xr-x 2 root root 4.0K Jul 3 13:15 killercoda
drwxr-xr-x 2 landscape landscape 4.0K May 18 12:59 landscape
-rw-rw-r-- 1 root utmp 292 Jun 17 06:26 lastlog
drwx----- 2 root root 4.0K Jun 17 06:25 private
-rw-r----- 1 syslog adm 149K Jul 3 14:10 syslog
-rw-r----- 1 syslog adm 363K Jun 20 08:54 syslog.1
drwxr-xr-x 2 root root 4.0K Jul 3 13:00 sysstat
drwxr-x--- 2 root adm 4.0K Jun 17 06:25 unattended-upgrades
-rw-rw-r-- 1 root utmp 7.5K Jul 3 13:00 wtmp

```

```

root@ubuntu:~$ ls -lh /var/log/ | sort -k5 -rh | head -8
-rw-r----- 1 syslog adm 363K Jun 20 08:54 syslog.1
-rw-r----- 1 syslog adm 155K Jun 20 08:53 kern.log.1
-rw-r----- 1 syslog adm 149K Jul 3 14:14 syslog
-rw-r--r-- 1 root root 127K Jun 20 08:54 dpkg.log.1
-rw-r----- 1 syslog adm 92K Jun 17 06:25 cloud-init.log
-rw-r----- 1 syslog adm 65K Jul 3 13:00 kern.log
-rw-r----- 1 root adm 59K Jul 3 13:00 dmesg
-rw-r----- 1 root adm 55K Jun 20 08:51 dmesg.0
root@ubuntu:~$ █

```

```

root@ubuntu:~$ wc -l /var/log/syslog
1253 /var/log/syslog
root@ubuntu:~$ wc -l /var/log/auth.log
42 /var/log/auth.log
root@ubuntu:~$ █

```

### Passo 3.2 - Monitorizar o Syslog em Tempo Real

**Procedimento:** O comando `tail -f /var/log/syslog` foi executado para seguir o registo em tempo real. Posteriormente, a monitorização foi filtrada para apenas exibir eventos críticos através do comando `tail -f /var/log/syslog | grep -i 'error\|warn\|fail'`.

**Evidência** do `tail -f` em execução com a filtragem aplicada.



```

root@ubuntu:~$ tail -f /var/log/syslog
2026-07-03T14:09:11.671810+00:00 ubuntu systemd[1]: phpsessionclean.service: Deactivated successfully.
2026-07-03T14:09:11.672402+00:00 ubuntu systemd[1]: Finished phpsessionclean.service - Clean php session files.
2026-07-03T14:10:21.589667+00:00 ubuntu systemd[1]: Starting sysstat-collect.service - system activity accounting tool...
2026-07-03T14:10:21.604296+00:00 ubuntu systemd[1]: sysstat-collect.service: Deactivated successfully.
2026-07-03T14:10:21.604880+00:00 ubuntu systemd[1]: Finished sysstat-collect.service - system activity accounting tool.
2026-07-03T14:15:01.200127+00:00 ubuntu CRON[1707]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
2026-07-03T14:17:01.207248+00:00 ubuntu CRON[1711]: (root) CMD (cd / && run-parts --report /etc/cron.hourly)
2026-07-03T14:20:39.010740+00:00 ubuntu systemd[1]: Starting sysstat-collect.service - system activity accounting tool...
2026-07-03T14:20:39.023188+00:00 ubuntu systemd[1]: sysstat-collect.service: Deactivated successfully.
2026-07-03T14:20:39.023950+00:00 ubuntu systemd[1]: Finished sysstat-collect.service - system activity accounting tool.
2026-07-03T14:25:01.223635+00:00 ubuntu CRON[1751]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
2026-07-03T14:28:51.587161+00:00 ubuntu systemd[1]: Starting fstrim.service - Discard unused blocks on filesystems from /etc/fstab...
2026-07-03T14:28:51.915047+00:00 ubuntu fstrim[1754]: /boot/efi: 98.2 MiB (102995968 bytes) trimmed on /dev/vda15
2026-07-03T14:28:51.916461+00:00 ubuntu fstrim[1754]: /boot: 761.9 MiB (798883840 bytes) trimmed on /dev/vda16
2026-07-03T14:28:51.917126+00:00 ubuntu fstrim[1754]: /: 13 GiB (13961256960 bytes) trimmed on /dev/vda1
2026-07-03T14:28:51.918112+00:00 ubuntu systemd[1]: fstrim.service: Deactivated successfully.
2026-07-03T14:28:51.919232+00:00 ubuntu systemd[1]: Finished fstrim.service - Discard unused blocks on filesystems from /etc/fstab.
2026-07-03T14:30:11.587089+00:00 ubuntu systemd[1]: Starting sysstat-collect.service - system activity accounting tool...
2026-07-03T14:30:11.596915+00:00 ubuntu systemd[1]: sysstat-collect.service: Deactivated successfully.
2026-07-03T14:30:11.598552+00:00 ubuntu systemd[1]: Finished sysstat-collect.service - system activity accounting tool.

```

```

2026-07-03T14:35:01.239361+00:00 ubuntu CRON[1764]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
2026-07-03T14:39:01.246529+00:00 ubuntu CRON[1767]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
2026-07-03T14:39:31.588811+00:00 ubuntu systemd[1]: Starting phpsessionclean.service - Clean php session files...
2026-07-03T14:39:31.663281+00:00 ubuntu systemd[1]: phpsessionclean.service: Deactivated successfully.
2026-07-03T14:39:31.664058+00:00 ubuntu systemd[1]: Finished phpsessionclean.service - Clean php session files.
2026-07-03T14:40:39.011420+00:00 ubuntu systemd[1]: Starting sysstat-collect.service - system activity accounting tool...
2026-07-03T14:40:39.022242+00:00 ubuntu systemd[1]: sysstat-collect.service: Deactivated successfully.
2026-07-03T14:40:39.023397+00:00 ubuntu systemd[1]: Finished sysstat-collect.service - system activity accounting tool.

```

Enquanto tail -f estava ativo, tentei abrir outro terminal para executar alguns comandos para gerar entradas no log para a seguir usar Ctrl+C para parar e analisa o output.

```

root@ubuntu:~$ $ tail -20 /var/log/syslog
$: command not found
root@ubuntu:~$ tail -f /var/log/syslog | grep -i 'error\|warn\|fail'

```

### Passo 3.3 - Investigar o Auth.log

**Procedimento:** Foram simuladas falhas de acesso através dos comandos su utilizador\_inexistente e sudo cat /etc/shadow. A investigação foi realizada com sudo tail -20 /var/log/auth.log para ver as últimas entradas e sudo grep 'Failed\|Invalid' /var/log/auth.log | wc -l para contabilizar as tentativas falhadas.

**Evidência:** sessao4\_passo3\_logs.png.

```

root@ubuntu:~$ su utilizador_inexistente
su: user utilizador_inexistente does not exist or the user entry does not contain all the required fields
root@ubuntu:~$ sudo cat /etc/shadow
root:$y$j9T$dLNqzr0VXNw8HducYFRGu/$r0TFdLeZwJrQbTIOkKeQRCTMcqu1IXWP3sj956RnBy9:20621:0:99999:7:::
daemon*:20591:0:99999:7:::
bin*:20591:0:99999:7:::
sys*:20591:0:99999:7:::
sync*:20591:0:99999:7:::
games*:20591:0:99999:7:::
man*:20591:0:99999:7:::
lp*:20591:0:99999:7:::
mail*:20591:0:99999:7:::
news*:20591:0:99999:7:::
uucp*:20591:0:99999:7:::
proxy*:20591:0:99999:7:::
www-data*:20591:0:99999:7:::
backup*:20591:0:99999:7:::
list*:20591:0:99999:7:::
irc*:20591:0:99999:7:::
_apt*:20591:0:99999:7:::
nobody*:20591:0:99999:7:::
systemd-networkd!:20591:0:99999:7:::
systemd-timesyncd!:20591:0:99999:7:::
dhcpcd!:20591:0:99999:7:::
messagebus!:20591:0:99999:7:::
syslog!:20591:0:99999:7:::
systemd-resolved!:20591:0:99999:7:::
uuidd!:20591:0:99999:7:::
tss!:20591:0:99999:7:::
sshd!:20591:0:99999:7:::
pollinate!:20591:0:99999:7:::

```

Ativar o Windows

Aceda a Definições para ativar o Windows.

```

tcpdump!:20591:0:99999:7:::
landscape!:20591:0:99999:7:::
fwupd-refresh!:20591:0:99999:7:::
polkitd!:20591:0:99999:7:::
ubuntu:$y$j9T$FQuXXe1h5xt18YZLBr/oY1$VXUIgPN674Xf.NxtgtQXY0sLJdTfnagpbchj6eyWm10:20621:0:99999:7:::
dnsmasq!:20624:0:99999:7:::
root@ubuntu:~$

```

Ativar o Windows

Aceda a Definições para ativar o Windows.

Depois, investiguei :

```

root@ubuntu:~$ sudo tail -20 /var/log/auth.log
2026-07-03T14:15:01.200250+00:00 ubuntu CRON[1706]: pam_unix(cron:session): session closed for user root
2026-07-03T14:17:01.204835+00:00 ubuntu CRON[1710]: pam_unix(cron:session): session opened for user root(uid=0) by
root(uid=0)
2026-07-03T14:17:01.215824+00:00 ubuntu CRON[1710]: pam_unix(cron:session): session closed for user root
2026-07-03T14:17:08.014512+00:00 ubuntu sshd[1713]: Accepted password for root from 10.244.6.35 port 56276 ssh2
2026-07-03T14:17:08.512446+00:00 ubuntu sshd[1716]: Accepted password for root from 10.244.6.35 port 56290 ssh2
2026-07-03T14:17:08.646807+00:00 ubuntu sshd[1719]: Accepted password for root from 10.244.6.35 port 56306 ssh2
2026-07-03T14:25:01.221805+00:00 ubuntu CRON[1750]: pam_unix(cron:session): session opened for user root(uid=0) by
root(uid=0)
2026-07-03T14:25:01.227285+00:00 ubuntu CRON[1750]: pam_unix(cron:session): session closed for user root
2026-07-03T14:35:01.233959+00:00 ubuntu CRON[1763]: pam_unix(cron:session): session opened for user root(uid=0) by
root(uid=0)
2026-07-03T14:35:01.239432+00:00 ubuntu CRON[1763]: pam_unix(cron:session): session closed for user root
2026-07-03T14:39:01.244686+00:00 ubuntu CRON[1766]: pam_unix(cron:session): session opened for user root(uid=0) by
root(uid=0)
2026-07-03T14:39:01.250269+00:00 ubuntu CRON[1766]: pam_unix(cron:session): session closed for user root
2026-07-03T14:45:01.255750+00:00 ubuntu CRON[1812]: pam_unix(cron:session): session opened for user root(uid=0) by
root(uid=0)
2026-07-03T14:45:01.260227+00:00 ubuntu CRON[1812]: pam_unix(cron:session): session closed for user root
2026-07-03T14:55:01.267230+00:00 ubuntu CRON[1855]: pam_unix(cron:session): session opened for user root(uid=0) by
root(uid=0)
2026-07-03T14:55:01.272268+00:00 ubuntu CRON[1855]: pam_unix(cron:session): session closed for user root
2026-07-03T14:55:48.733030+00:00 ubuntu sudo: root : TTY=pts/2 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/cat /
etc/shadow
2026-07-03T14:55:48.734547+00:00 ubuntu sudo: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=0
)
2026-07-03T14:55:48.737886+00:00 ubuntu sudo: pam_unix(sudo:session): session closed for user root
2026-07-03T14:59:03.511793+00:00 ubuntu sudo: root : TTY=pts/2 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/tail
-20 /var/log/auth.log
root@ubuntu:~$

```

Ativar o Windows

Aceda a Definições para ativar o Windows.

```

root@ubuntu:~$ sudo grep 'Failed\\|Invalid\\|FAILED' /var/log/auth.log | tail -10
2026-07-03T15:00:40.028394+00:00 ubuntu sudo:      root : TTY=pts/2 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/grep
Failed\\|Invalid\\|FAILED /var/log/auth.log
root@ubuntu:~$ sudo grep 'Failed\\|Invalid' /var/log/auth.log | wc -l
2
root@ubuntu:~$ sudo grep 'Accepted' /var/log/auth.log | tail -5
2026-07-03T14:12:06.446087+00:00 ubuntu sshd[1701]: Accepted password for root from 10.244.4.102 port 34114 ssh2
2026-07-03T14:17:08.014512+00:00 ubuntu sshd[1713]: Accepted password for root from 10.244.6.35 port 56276 ssh2
2026-07-03T14:17:08.512446+00:00 ubuntu sshd[1716]: Accepted password for root from 10.244.6.35 port 56290 ssh2
2026-07-03T14:17:08.646807+00:00 ubuntu sshd[1719]: Accepted password for root from 10.244.6.35 port 56306 ssh2
2026-07-03T15:01:18.270635+00:00 ubuntu sudo:      root : TTY=pts/2 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/grep
Accepted /var/log/auth.log
root@ubuntu:~$

```

### Passo 3.4 - Criar Relatório de Incidente

**Procedimento:** A pasta ~/relatorios/sessao4 foi criada com mkdir -p e o ficheiro incidente.txt foi gerado utilizando echo com redirecionamento >> para documentar a data, o analista, o diagnóstico do processo, as ações efetuadas e os logs de autenticação. O conteúdo final foi verificado com cat ~/relatorios/sessao4/incidente.txt.

**Evidência** do conteúdo do relatório incidente.txt.

```

root@ubuntu:~$ mkdir -p ~/relatorios/sessao4
root@ubuntu:~$ echo "=== RELATÓRIO DE INCIDENTE – SESSÃO 4 ===" >
~/relatorios/sessao4/incidente.txt
bash: syntax error near unexpected token `newline'
bash: /root/relatorios/sessao4/incidente.txt: No such file or directory
root@ubuntu:~$ echo "Data: $(date)" >> ~/relatorios/sessao4/incidente.txt
root@ubuntu:~$ echo "Analista: $(whoami)" >> ~/relatorios/sessao4/incidente.txt
root@ubuntu:~$ echo "" >> ~/relatorios/sessao4/incidente.txt
root@ubuntu:~$ echo "--- DIAGNÓSTICO: PROCESSOS ---" >>
~/relatorios/sessao4/incidente.txt
bash: syntax error near unexpected token `newline'
bash: /root/relatorios/sessao4/incidente.txt: Permission denied
root@ubuntu:~$ ps aux --sort=-%cpu | head -10 >> ~/relatorios/sessao4/incidente.txt
root@ubuntu:~$ echo "--- AÇÕES TOMADAS ---" >> ~/relatorios/sessao4/incidente.txt
root@ubuntu:~$ echo "1. Processo identificado: PID=____ Comando=____" >>
~/relatorios/sessao4/incidente.txt
bash: syntax error near unexpected token `newline'
bash: /root/relatorios/sessao4/incidente.txt: Permission denied
root@ubuntu:~$ echo "2. SIGTERM enviado: kill PID → ignorado pelo processo" >>
~/relatorios/sessao4/incidente.txt
bash: syntax error near unexpected token `newline'
bash: /root/relatorios/sessao4/incidente.txt: Permission denied
root@ubuntu:~$ echo "3. SIGKILL enviado: kill -9 PID → processo terminado" >>
~/relatorios/sessao4/incidente.txt
bash: syntax error near unexpected token `newline'
bash: /root/relatorios/sessao4/incidente.txt: Permission denied
root@ubuntu:~$ echo "--- LOGS DE AUTENTICAÇÃO ---" >> ~/relatorios/sessao4/incidente.txt
root@ubuntu:~$ sudo grep 'Failed' /var/log/auth.log | tail -5 >>
~/relatorios/sessao4/incidente.txt
bash: syntax error near unexpected token `newline'
bash: /root/relatorios/sessao4/incidente.txt: Permission denied

```

Ativar o Windows  
Aceda a Definições para ativar o Windows.

```

root@ubuntu:~$ cat ~/relatorios/sessao4/incidente.txt
Data: Fri Jul 3 15:05:45 UTC 2026
Analista: root

USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      1285  0.2  0.6 1231188 12632 ?        S<l  13:51   0:10 /bin/runtime-info-service
root      1541  0.1  2.8 1243688 54832 ?        SNl  13:51   0:07 /opt/theia/node /opt/theia/node_modules/@theia/c
ore/lib/node/messaging/ipc-bootstrap --watchOptions={}
root      1248  0.1  5.1 11761708 100684 ?       SNl  13:51   0:05 /opt/theia/node /opt/theia/browser-app/src-gen/b
ackend/main.js /root --hostname=0.0.0.0 --port 40205
root       1  0.0  0.6  22104 13240 ?        Ss   13:51   0:04 /sbin/init
root      628  0.0  2.4 1792748 48212 ?        Ssl  13:51   0:02 /usr/bin/containerd
root      910  0.0  3.9 1761524 76856 ?        Ssl  13:51   0:00 /usr/bin/dockerd -H fd:// --containerd=/run/cont
ainerd/containerd.sock
root       17  0.0  0.0      0     0 ?        I    13:51   0:00 [rcu_preempt]
root      336  0.0  1.4 288952 27292 ?        Ssl  13:51   0:00 /sbin/multipathd -d -s
root       10  0.0  0.0      0     0 ?        I<   13:51   0:00 [kworker/0:0H-kblockd]

--- AÇÕES TOMADAS ---
--- LOGS DE AUTENTICAÇÃO ---
root@ubuntu:~$

```

## Questões Finais

- **Como configurarias o sistema para ser alertado automaticamente quando o %CPU de um processo ultrapassa 90%?**
  - A configuração seria efetuada através de ferramentas de monitorização e gestão de alertas (como *Prometheus* com *Alertmanager* ou *Zabbix*), ou via scripts agendados em cron que verificam periodicamente o consumo de CPU e disparam notificações.
- **O comando `journalctl` é uma alternativa moderna ao `tail -f`. Experimenta: `journalctl -f`. Que diferenças notas?**
  - O `journalctl` acede a logs centralizados e estruturados pelo *systemd* (formatados em binário), oferecendo filtros avançados por tempo, unidade ou prioridade. O `tail -f` limita-se à leitura linear de ficheiros de texto simples.
- **Em que situação usarias `grep -v 'CRON' /var/log/syslog` e o que eliminaria do output?**
  - Esta filtragem seria utilizada para remover o ruído gerado por tarefas automáticas agendadas (*CRON*), permitindo uma análise mais focada em eventos operacionais não automáticos. Eliminaría todas as linhas do *output* que contivessem o termo "CRON".